

Lab -08

Task no 1:

hospital database:

```
1 CREATE TABLE Doctors (
2     doctor_id INT PRIMARY KEY,
3     doctor_name VARCHAR(100),
4     specialization VARCHAR(100),
5     experience_years INT
6 );
7
8 CREATE TABLE Patients (
9     patient_id INT PRIMARY KEY,
10    patient_name VARCHAR(100),
11    age INT,
12    city VARCHAR(100)
13 );
14
15 CREATE TABLE Appointments (
16     appointment_id INT PRIMARY KEY,
17     doctor_id INT,
18     patient_id INT,
19     appointment_date DATE,
20     diagnosis VARCHAR(255),
21     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
22     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
23 );
24
25 -- Doctors
26 INSERT INTO Doctors VALUES
27 (1, 'Dr. Alice Smith', 'Cardiology', 15),
28 (2, 'Dr. Bob Johnson', 'Neurology', 20),
29 (3, 'Dr. Clara Oswald', 'Dermatology', 10),
30 (4, 'Dr. David Lee', 'Orthopedics', 25),
31 (5, 'Dr. Eva Brown', 'Pediatrics', 18);
32
33 -- Patients
34 INSERT INTO Patients VALUES
35 (1, 'John Doe', 30, 'New York'),
36 (2, 'Jane Roe', 45, 'Los Angeles'),
37 (3, 'Michael Chan', 25, 'Chicago'),
38 (4, 'Emily Davis', 35, 'Houston'),
39 (5, 'Sarah Miller', 40, 'Phoenix'),
40 (6, 'Chris Wilson', 50, 'New York');
41
42 -- Appointments
43 INSERT INTO Appointments VALUES
44 (1, 1, 1, '2023-01-10', 'Hypertension'),
45 (2, 2, 2, '2023-01-05', 'Migraine'),
46 (3, 2, 3, '2023-02-15', 'Seizure'),
47 (4, 3, 4, '2023-03-12', 'Acne'),
48 (5, 4, 5, '2023-04-20', 'Fracture'),
49 (6, 5, 6, '2023-05-25', 'Flu'),
50 (7, 2, 6, '2023-06-01', 'Headache'),
51 (8, 1, 3, '2023-07-15', 'Chest Pain');
```

Query no1:

```
51 --Query 1: Doctors with experience above average
52 SELECT doctor_name
53 FROM Doctors
54 WHERE experience_years > (
55     SELECT AVG(experience_years) FROM Doctors
56 );
57
58 --Query 2: Patient with the earliest appointment
59
60 doctor_name
61
62 Dr. Bob Johnson
63
64 Dr. David Lee
65
66 Dr. Eva Brown
```

Query no2:

```
57 --Query 2: Patient with the earliest appointment
58 SELECT p.patient_name
59 FROM Appointments a
60 JOIN Patients p ON a.patient_id = p.patient_id
61 WHERE a.appointment_date = (
62     SELECT MIN(appointment_date) FROM Appointments
63 );
```

! patient_name

Jane Roe

Query no 3:

```
64 --Query 3: Doctors with appointments with more than one patient
65 SELECT d.doctor_name
66 FROM Appointments a
67 JOIN Doctors d ON a.doctor_id = d.doctor_id
68 GROUP BY d.doctor_id, d.doctor_name
69 HAVING COUNT(DISTINCT a.patient_id) > 1;
```

! doctor_name

Dr. Alice Smith

Dr. Bob Johnson

Query no 4:

```
70 --Query 4: Patients older than the average age of patients treated by the same doctor
71 SELECT DISTINCT p.patient_name
72 FROM Patients p
73 JOIN Appointments a ON p.patient_id = a.patient_id
74 WHERE p.age > (
75     SELECT AVG(p2.age)
76     FROM Appointments a2
77     JOIN Patients p2 ON a2.patient_id = p2.patient_id
78     WHERE a2.doctor_id = a.doctor_id
79 );
```

! patient_name

John Doe

Jane Roe

Chris Wilson

Query no 5:

```
80 --Query 5: Second most experienced doctor
81 SELECT doctor_name
82 FROM Doctors
83 WHERE experience_years = (
84     SELECT MAX(experience_years)
85     FROM Doctors
86     WHERE experience_years < (
87         SELECT MAX(experience_years) FROM Doctors
88     )
89 );
90
```

! doctor_name

Dr. Bob Johnson

Query no6:

```
90 --Query 6: Insert new appointment for patient with latest appointment with a specific doctor (c
91 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date, diagnosis)
92 SELECT 9, doctor_id, patient_id, '2025-10-19', 'Follow-up'
93 FROM Appointments
94 WHERE doctor_id = 2
95 ORDER BY appointment_date DESC
96 LIMIT 1;
97
98
99
```

! 9	doctor_id	patient_id	'2025-10-19'	'Follow-up'
9	2	6	2025-10-19	Follow-up

Task no 2:

(online shopping platform)

```
1 CREATE TABLE Products (  
2   product_id INT PRIMARY KEY,  
3   product_name VARCHAR(100),  
4   category VARCHAR(100),  
5   price DECIMAL(10, 2),  
6   stock INT  
7 );  
8 CREATE TABLE Customers (  
9   customer_id INT PRIMARY KEY,  
10  customer_name VARCHAR(100),  
11  city VARCHAR(100),  
12  age INT  
13 );  
14 CREATE TABLE Orders (  
15  order_id INT PRIMARY KEY,  
16  customer_id INT,  
17  product_id INT,  
18  order_date DATE,  
19  quantity INT,  
20  total_amount DECIMAL(10, 2),  
21  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
22  FOREIGN KEY (product_id) REFERENCES Products(product_id)  
23 );  
24 INSERT INTO Products VALUES  
25 (1, 'Laptop', 'Electronics', 1000.00, 20),  
26 (2, 'Headphones', 'Electronics', 200.00, 50),  
27 (3, 'Chair', 'Furniture', 150.00, 30),  
28 (4, 'Desk', 'Furniture', 300.00, 25),  
29 (5, 'Smartphone', 'Electronics', 800.00, 40);  
30 INSERT INTO Customers VALUES  
31 (1, 'Alice Johnson', 'New York', 28),  
32 (2, 'Bob Smith', 'Los Angeles', 35),  
33 (3, 'Carol White', 'New York', 42),  
34 (4, 'David Brown', 'Chicago', 30),  
35 (5, 'Eva Green', 'Los Angeles', 26);  
36 INSERT INTO Orders VALUES  
37 (1, 1, 1, '2023-01-05', 1, 1000.00),  
38 (2, 2, 2, '2023-01-10', 2, 400.00),  
39 (3, 3, 1, '2023-01-15', 1, 1000.00),  
40 (4, 4, 3, '2023-01-20', 2, 300.00),
```

Query no 1:

```
45 --1. Find the names of products whose price is higher than the average price of all products  
46 SELECT product_name  
47 FROM Products  
48 WHERE price > (  
49   SELECT AVG(price) FROM Products  
50 );  
51
```

product_name

Laptop

Smartphone

Query no 2:

```
51 --2. Get the name of the customer who placed the first (earliest) order
52 SELECT c.customer_name
53 FROM Orders o
54 JOIN Customers c ON o.customer_id = c.customer_id
55 WHERE o.order_date = (
56     SELECT MIN(order_date) FROM Orders
57 );
58 --3. Find names of products that have been ordered by more than one customer
```

! customer_name

Alice Johnson

Query no 2:

```
58 --3. Find names of products that have been ordered by more than one customer
59 SELECT p.product_name
60 FROM Orders o
61 JOIN Products p ON o.product_id = p.product_id
62 GROUP BY p.product_id, p.product_name
63 HAVING COUNT(DISTINCT o.customer_id) > 1;
64
```

! product_name

Laptop

Headphones

Smartphone

Query no 3:

```
58 --3. Find names of products that have been ordered by more than one customer
59 SELECT p.product_name
60 FROM Orders o
61 JOIN Products p ON o.product_id = p.product_id
62 GROUP BY p.product_id, p.product_name
63 HAVING COUNT(DISTINCT o.customer_id) > 1;
64
```

! product_name

Laptop

Headphones

Smartphone

Query no 4:

```
65 SELECT c.customer_name
66 FROM Customers c
67 JOIN (
68     SELECT customer_id, SUM(total_amount) AS total_spent
69     FROM Orders
70     GROUP BY customer_id
71 ) t ON c.customer_id = t.customer_id
72 WHERE t.total_spent > (
73     SELECT AVG(t2.total_spent)
74     FROM Customers c2
75     JOIN (
76         SELECT customer_id, SUM(total_amount) AS total_spent
77         FROM Orders
78         GROUP BY customer_id
79     ) t2 ON c2.customer_id = t2.customer_id
80     WHERE c2.city = c.city
81 );
82 --5. Find the product with the second highest price
```

customer_name

Alice Johnson

Bob Smith

Query no 5:

```
82 --5. Find the product with the second highest price
83 SELECT product_name
84 FROM Products
85 WHERE price = (
86     SELECT MAX(price)
87     FROM Products
88     WHERE price < (
89         SELECT MAX(price) FROM Products
90     )
91 );
```

product_name

Smartphone

Task no 3:

```
1 -- Orchestras table
2 CREATE TABLE orchestras (
3   id INT PRIMARY KEY,
4   name VARCHAR(100),
5   rating DECIMAL(3,1),
6   city VARCHAR(100),
7   country VARCHAR(100),
8   founded_year INT
9 );
10 -- Concerts table
11 CREATE TABLE concerts (
12   id INT PRIMARY KEY,
13   city VARCHAR(100),
14   country VARCHAR(100),
15   year INT,
16   rating DECIMAL(3,1),
17   orchestra_id INT,
18   FOREIGN KEY (orchestra_id) REFERENCES orchestras(id)
19 );
20 -- Members table
21 CREATE TABLE members (
22   id INT PRIMARY KEY,
23   name VARCHAR(100),
24   instrument VARCHAR(50),
25   wage DECIMAL(10,2),
26   experience INT,
27   orchestra_id INT,
28   FOREIGN KEY (orchestra_id) REFERENCES orchestras(id)
29 );
30 INSERT INTO orchestras VALUES
31 (1, 'Symphony Stars', 8.5, 'Vienna', 'Austria', 1958),
32 (2, 'Chamber Orchestra', 7.8, 'Berlin', 'Germany', 1968),
33 (3, 'Harmonic Ensemble', 8.9, 'Paris', 'France', 1988),
34 (4, 'Classic Vibes', 7.2, 'London', 'UK', 1975),
35 (5, 'Modern Strings', 9.1, 'New York', 'USA', 1990);
36
37 INSERT INTO concerts VALUES
38 (1, 'Paris', 'France', 2013, 8.0, 3),
39 (2, 'Vienna', 'Austria', 2012, 9.0, 1),
40 (3, 'Berlin', 'Germany', 2013, 7.5, 2),
41 (4, 'London', 'UK', 2011, 6.8, 4),
42 (5, 'New York', 'USA', 2013, 9.2, 5),
43 (6, 'Rome', 'Italy', 2014, 7.9, 3);
44
45 INSERT INTO members VALUES
46 (1, 'Alice', 'violin', 3500.00, 12, 1),
47 (2, 'Bob', 'cello', 4000.00, 15, 1),
48 (3, 'Charlie', 'violin', 3200.00, 8, 2),
49 (4, 'Diana', 'Flute', 3000.00, 11, 3),
50 (5, 'Edward', 'viola', 4500.00, 9, 3),
51 (6, 'Fiona', 'violin', 3600.00, 13, 3),
52 (7, 'George', 'cello', 4200.00, 14, 4),
53 (8, 'Hannah', 'harp', 3100.00, 7, 5),
54 (9, 'Ivan', 'violin', 2800.00, 10, 5),
55 (10, 'Jane', 'piano', 5000.00, 16, 5),
56 (11, 'Karl', 'violin', 3000.00, 14, 3);
```

Query no 1:

```
57 --1. Orchestras from cities that hosted a concert in 2013
58 SELECT DISTINCT o.name
59 FROM orchestras o
60 WHERE o.city IN (
61   SELECT DISTINCT city
62   FROM concerts
63   WHERE year = 2013
64 );
```

! name

Chamber Orchestra

Harmonic Ensemble

Modern Strings

Query no 2:

```
65 --2. Members with >10 years of experience in orchestras rated ≥ 8.0
66 SELECT m.name, m.instrument
67 FROM members m
68 JOIN orchestras o ON m.orchestra_id = o.id
69 WHERE m.experience > 10
70 AND o.rating >= 8.0;
```

! name	instrument
Alice	violin
Bob	cello
Diana	flute
Fiona	violin
Jane	piano

Query no 3:

```
71 --3. Members with wage > average wage of all violinists
72 SELECT name, instrument
73 FROM members
74 WHERE wage > (
75     SELECT AVG(wage)
76     FROM members
77     WHERE instrument = 'violin'
78 );
79 --4. Orchestras founded after 'Chamber Orchestra' and with rating > 7.5
```

! name	instrument
Alice	violin
Bob	cello
Edward	viola
Fiona	violin
George	cello

Query no 4:

```
79 --4. Orchestras founded after 'Chamber Orchestra' and with rating > 7.5
80 SELECT name
81 FROM orchestras
82 WHERE founded_year > (
83     SELECT founded_year
84     FROM orchestras
85     WHERE name = 'Chamber Orchestra'
86 )
87 AND rating > 7.5;
```

name
Harmonic Ensemble
Modern Strings

Query no 5:

```
88 --5. Orchestras with more members than average
89 SELECT o.name, COUNT(m.id) AS total_members
90 FROM orchestras o
91 JOIN members m ON o.id = m.orchestra_id
92 GROUP BY o.id, o.name
93 HAVING COUNT(m.id) > (
94     SELECT AVG(member_count)
95     FROM (
96         SELECT orchestra_id, COUNT(*) AS member_count
97         FROM members
98         GROUP BY orchestra_id
99     ) AS member_counts
100 );
101
```

name	total_members
Harmonic Ensemble	4
Modern Strings	3